Cosmos3: World Foundation Models for Perception, Reasoning, Simulation, and Action

NVIDIA[1]

Abstract

[1]A detailed list of contributors and acknowledgments can be found in **??** of this paper.

Contents

1. Introduction

To build intelligent agents that can operate in the physical world, we must endow them with models of the world itself. Such world models have two fundamentally coupled facets: understanding and generation. Understanding enables an agent to infer latent state, semantics, and dynamics from partial observations, while generation enables it to predict and simulate plausible futures under those dynamics. Prior work has largely treated these capabilities in isolation—discriminative models for perception and reasoning, and generative models for prediction and synthesis. We argue that this separation is limiting: true understanding requires the ability to generate the evolution of the world, and coherent generation requires internalizing its underlying structure. Unifying these two facets into a single scalable framework is therefore essential for Physical AI.

We introduce Cosmos3, a world foundation model that jointly models understanding and generation through a Mixture-of-Transformer (MoT) architecture. Cosmos3 comprises two tightly coupled components: a reasoner tower and a generator tower. The reasoner is a vision-language model that interprets multimodal observations—particularly videos—and maps raw sensory inputs into structured latent representations of the world, supporting tasks such as question answering, spatial and temporal reasoning, and event understanding. The generator, in turn, acts as a world simulator, synthesizing future observations and actions from these latent states to produce temporally consistent video and action sequences, enabling long-horizon rollouts of plausible world dynamics. The two towers are optimized for complementary objectives, and by sharing representations, Cosmos3 enables inferred world states from the reasoner to directly guide generation, grounding simulation in structured understanding (Figure 1).

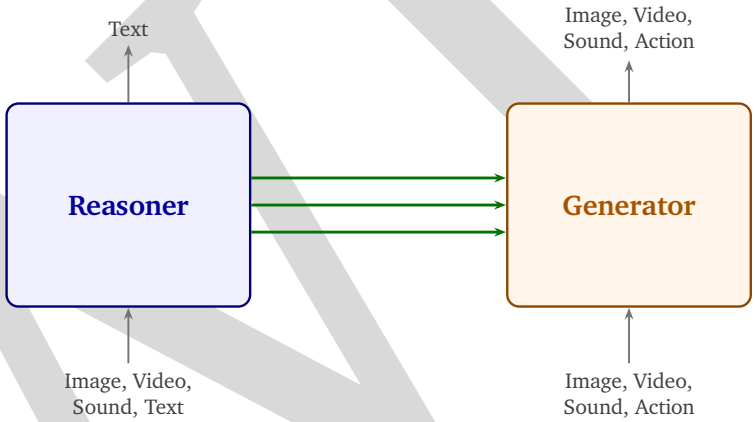

Figure 1: Architecture of Cosmos3, comprising a reasoner and a generator linked via shared latent representations. The reasoner maps multimodal inputs (image, video, audio, text) into latent world states for perception and reasoning, while the generator conditions on these states to produce multimodal outputs (image, video, audio, action). This unidirectional interface grounds generation in structured understanding and enables long-horizon simulation of world dynamics.

A defining property of Physical AI is that agents actively change the state of the world, creating a closed-loop interaction between perception, reasoning, and control. This requires treating actions as first-class entities alongside language and vision. Cosmos3 adopts an omni-modal formulation over text, video, and action, supporting flexible conditioning and generation across these modalities. It learns a unified representation over heterogeneous action spaces spanning multiple embodiments, including human body poses, robot end-effectors, vehicle trajectories, and camera motions. By modeling actions within the same latent space as observations, Cosmos3 captures how actions transform world states, enabling action-conditioned prediction and counterfactual reasoning.

By jointly modeling language, video, and action, Cosmos3 serves as a general-purpose backbone for Physical AI, subsuming a wide range of model classes within a single framework (Table 1). Depending on the input-output

configuration, it can operate as a vision-language model (video/text → text) for understanding, a video generator (video/text → video) for synthesis, a world model (action/video/text → video) for predicting environment dynamics, and a policy model (video/text → action) for action generation. More generally, Cosmos3 functions as a unified world-action model (video/action/text → video & action), modeling both the evolution of the world and the impact of actions. This flexibility supports applications such as video understanding, synthetic data generation, and action-conditioned video synthesis. Cosmos3 can also directly serve as a policy model by providing a structured prior over world dynamics and action effects. Notably, all these capabilities are realized within a single architecture and shared set of parameters. By unifying perception, prediction, and action modeling, Cosmos3 reduces reliance on task-specific pipelines and enables scalable learning through shared representations and multi-task supervision.

Input Modality	Output Modality	Application
Video | Text	Text	Vision Language Model
Video | Text	Video	Video Generator
Action | Video | Text	Video	World Model
Video | Text	Action	Policy Model
Video | Action | Text	Video & Action	World Action Model

Table 1: Taxonomy of Physical AI models defined by input and output modalities. Cosmos3 unifies these model classes—including vision-language models, video generators, world models, policy models, and world-action models—within a single architecture by supporting flexible multimodal conditioning and generation over video, text, and action.

We develop Cosmos3 as a general-purpose backbone for Physical AI that can be adapted to different applications and embodiments. Since agents may perceive and interact with the world in different ways, it is necessary to allow task- and embodiment-specific specialization on top of a shared model. Cosmos3 provides such a starting point by modeling general world dynamics and action priors, while remaining amenable to downstream adaptation. In practice, the model can be post-trained on target data to support different roles, including understanding, world modeling, policy learning, and joint world-action modeling. This adaptation does not require changes to the model architecture, allowing specialization through data while retaining a common representation of the world, thanks to Cosmos3's omni-model design.

2. Model Architecture

2.1. Mixture-of-Transformers (MoT) Architecture

Cosmos3 is built upon a **Mixture-of-Transformers (MoT)** architecture that unifies multimodal understanding and generation within a single model. Rather than employing separate networks for perception and generation, MoT decomposes each transformer layer into two *specialized yet co-trained* sub-networks: an **Reasoner Tower** for multimodal comprehension, and a **Generator Tower** for visual synthesis. Both towers mirror the same block structure and are co-initialized from the weights of a pretrained Vision-Language Model (VLM), enabling Cosmos3 to inherit strong language grounding and visual reasoning capabilities while simultaneously learning to generate high-fidelity video.

The two towers process distinct but complementary token subsequences that together constitute each training sample (see Fig. 2). The Reasoner Tower processes the *autoregressive (AR) subsequence*, encompassing all conditioning tokens: text and video. The Generator Tower processes the *diffusion (DF) subsequence*, comprising the noisy video and action latents that are iteratively denoised during generation. The two subsequences are concatenated and processed jointly within every transformer layer, governed by an asymmetric attention

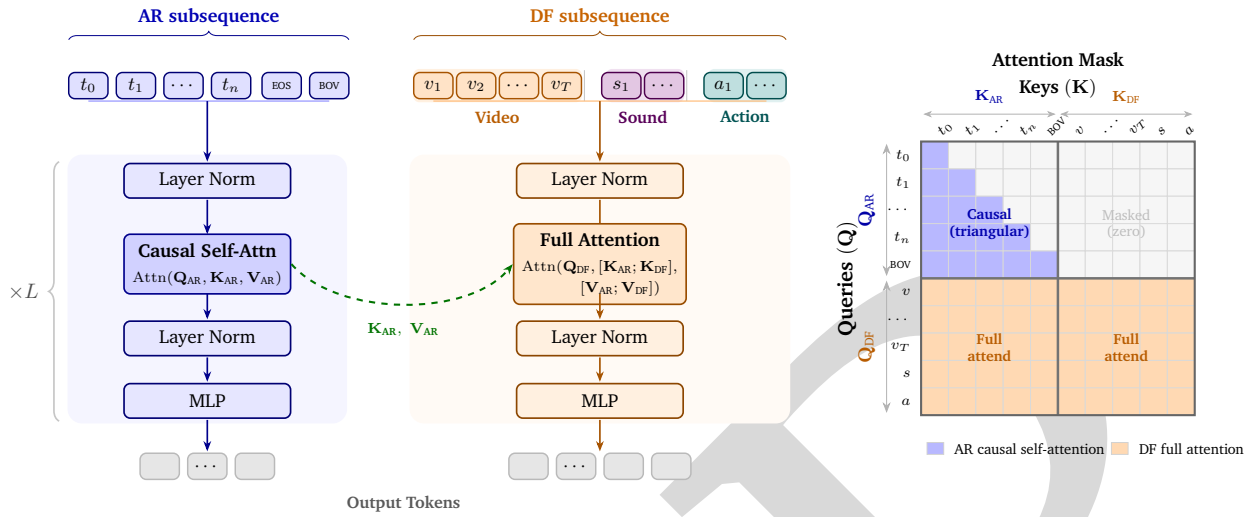

Figure 2: **Mixture-of-Transformers (MoT) architecture.** Within each transformer layer, the Reasoner Tower (AR, blue) applies causal self-attention over conditioning tokens (text, clean frames), while the Generator Tower (DF, orange) applies full bidirectional attention over the concatenated AR+DF context. The two towers carry fully independent weight matrices (**W**) and layer norms, all co-initialized from a pretrained LLM (such as Qwen3-VL). The dashed green arrow shows how AR keys and values are shared with the DF attention. The inset (top right) illustrates the corresponding attention masks: triangular (causal) for AR and full for DF.

scheme that reflects the distinct computational role of each tower.

2.1.1. Dual-Tower Layer Structure

At the heart of MoT is the observation that understanding and generation impose fundamentally different inductive biases on a transformer layer. Rather than forcing a single set of projection weights to serve both roles, each MoT layer maintains *fully independent weight matrices* for the two towers. Concretely, within each transformer layer ℓ, the Reasoner Tower has its own query, key, value, and output projections $\{\mathbf{W}^Q_{\text{AR}}, \mathbf{W}^K_{\text{AR}}, \mathbf{W}^V_{\text{AR}}, \mathbf{W}^O_{\text{AR}}\}$, its own feed-forward network MLP_{AR}, and its own pre- and post-attention layer norms. The Generator Tower has a fully separate, identically sized set of weights $\{\mathbf{W}^Q_{\text{DF}}, \mathbf{W}^K_{\text{DF}}, \mathbf{W}^V_{\text{DF}}, \mathbf{W}^O_{\text{DF}}\}$, MLP_{DF}, and layer norms. Both sets are initialized from the same pretrained language model.

This "twin-parameter" design means the total trainable parameter count of a Cosmos3 MoT model is approximately twice that of the underlying VLM backbone (for the transformer body), while the overall computational graph per forward pass processes the AR and DF subsequences in a single fused kernel call for efficiency.

2.1.2. Dual-Stream Attention

The asymmetric attention scheme determines how tokens in each tower attend to one another. Reflecting their distinct roles, the two towers employ structurally different attention patterns within every layer:

Reasoner Tower (AR).
Tokens in the AR subsequence attend with *causal self-attention*, attending only to preceding tokens in the same sequence. This preserves the autoregressive property inherited from the VLM backbone, allowing the model to perform coherent contextual reasoning over conditioning inputs:

$$\mathbf{O}_{\text{AR}} = \text{Attn}_{\text{causal}}\big(\mathbf{Q}_{\text{AR}},\ \mathbf{K}_{\text{AR}},\ \mathbf{V}_{\text{AR}}\big). \tag{1}$$

Table 2: **Cosmos3 MoT model variants.** All models share the dual-tower MoT architecture. "LLM Layers" refers to the number of transformer decoder layers; each layer carries independent parameter sets for the Understanding and Generation Towers.

Variant	LLM Layers	Hidden Dim	Attn Heads	KV Heads	Head Dim	FFN Dim
Nano	36	4096	32	8	128	12288
Super	64	5120	64	8	128	25600

Generator Tower (DF).
Tokens in the DF subsequence attend with *full bidirectional attention* over the union of AR and DF tokens. This allows every diffusion token to freely attend to the complete conditioning context produced by the Reasoner Tower, as well as to all other diffusion tokens in the sequence:

$$\mathbf{O}_{\text{DF}} = \text{Attn}_{\text{full}}\big(\mathbf{Q}_{\text{DF}},\ [\mathbf{K}_{\text{AR}};\ \mathbf{K}_{\text{DF}}],\ [\mathbf{V}_{\text{AR}};\ \mathbf{V}_{\text{DF}}]\big), \tag{2}$$

where $[\cdot\,;\cdot]$ denotes concatenation along the sequence dimension. Crucially, AR tokens are never updated based on DF tokens, preserving causal integrity of the conditioning pathway.

2.1.3. Two-Way Attention Decomposition

A naïve implementation of the scheme above would require a bespoke attention mask over the interleaved AR+DF sequence. This mask is asymmetric—triangular for the AR block but full for the DF block—and is unsupported by standard FlashAttention kernels **??**. We therefore adopt a textbftwo-way decomposition that replaces the single custom attention call with two standard attention operations per layer:

1. **Causal self-attention on AR:** $\text{Attn}_{\text{causal}}(\mathbf{Q}_{\text{AR}},\ \mathbf{K}_{\text{AR}},\ \mathbf{V}_{\text{AR}})$, implemented as a standard upper-triangular masked attention over the AR subsequence alone.
2. **Full attention from DF to AR + DF:** $\text{Attn}_{\text{full}}(\mathbf{Q}_{\text{DF}},\ [\mathbf{K}_{\text{AR}};\mathbf{K}_{\text{DF}}],\ [\mathbf{V}_{\text{AR}};\mathbf{V}_{\text{DF}}])$, implemented as unmasked full attention with the AR and DF key-value buffers concatenated.

This decomposition keeps AR and DF token buffers disjoint in GPU memory, requires no attention-output merging (outputs are simply concatenated and passed to the subsequent feedforward layer), and is fully compatible with FlashAttention.

2.1.4. VLM Backbones

Cosmos3 is trained at four model scales, spanning a wide range of computational budgets from on-device deployment to large datacenter inference. All variants are initialized from the pretrained LLM model and follow the MoT architecture described above. Tab. 2 summarizes the key architectural hyperparameters for each variant.

Nano. [TODO: description.]

Super. [TODO: description.]

2.2. Position Embedding

Cosmos3 is an inherently multimodal model that processes video, text, audio, and actions within a single unified architecture. Supporting these modalities jointly calls for a positional embedding scheme that simultaneously handles 3D spatio-temporal structure for video tokens, 1D structure for text, audio, and action tokens, varying spatial resolutions, and multiple frame rates within the same training batch. We use 3D Multimodal RoPE (MRoPE) Bai et al. (2025) to meet these requirements.

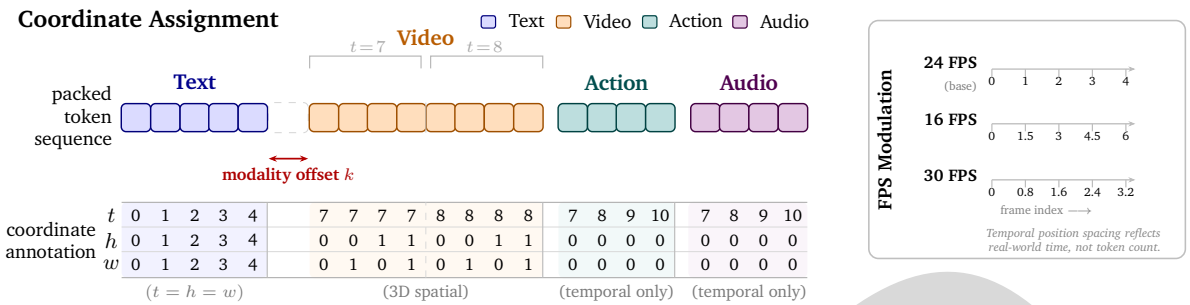

Figure 3: **Illustrative Coordinate assignment under 3D MRoPE. Left:** A packed token sequence containing text, video (two frames, 2×2 spatial grid each), action, and audio tokens. Each token receives a (t, h, w) triplet. Text tokens use $t = h = w$; video tokens vary on all three axes; action and audio tokens use temporal coordinates only ($h = w = 0$). A modality offset k separates the text and vision temporal ranges. **Right (inset):** FPS modulation maps frame indices to scaled temporal positions so that equal real-world durations occupy equal position ranges at 16, 24, and 30 FPS.

2.2.1. 3D M-RoPE

Core mechanism. Standard 1D RoPE was designed for sequential text where position is a single index. Applying it to video requires flattening the spatial dimensions, which destroys spatial locality — adjacent pixels in 2D space appear distant after flattening, breaking RoPE's assumption that nearby positions should have high attention scores. MRoPE addresses this by partitioning each attention head's embedding dimension into three non-overlapping subspaces assigned to time (t), height (h), and width (w). An independent rotation is applied to each subspace based solely on its corresponding positional index, so two tokens that are spatially close remain close in the rotated representation regardless of their sequence position. We use an interleaved (stride-3) channel assignment, ensuring each subspace receives a mixture of low and high frequencies. For a language backbone with head dimension 128, the per-half-dimension allocation is 24 channels for t, 20 for h, and 20 for w, with a base theta of 5×10^6.

Unified coordinate assignment. All tokens in the packed sequence share a single (t, h, w) coordinate system, illustrated in Figure 3. Text tokens set $t = h = w$ to the same monotonically increasing value, collapsing to standard 1D RoPE behavior. Video tokens vary across all three axes: t advances with the temporal latent frame index, while h and w tile over the spatial grid $(0 \ldots H-1,\ 0 \ldots W-1)$ independently per frame. Image tokens are treated as single-frame videos and vary only in (h, w). Audio and action tokens vary only in t, with $h = w = 0$. Spatial indices are reset to zero at the start of each vision segment, so the model treats h and w as absolute within-frame coordinates rather than positions in the global sequence.

Spatial reset. By default, we reset the spatial (h, w) indices to start from zero for each vision segment, giving the model an absolute spatial position within each frame. This decouples spatial positioning from sequence length — the same patch location receives the same spatial embedding regardless of how many text, audio, or action tokens precede it in the packed sequence. This is particularly important for image editing tasks where spatial consistency across generation steps is critical.

FPS modulation. Training videos span a wide range of frame rates, and Cosmos3 supports audio and action modalities with their own native sampling rates. To make the model aware of these differing rates at training time — and to provide explicit control over generation speed at inference — we modulate the temporal MRoPE indices based on each modality's tokens-per-second (tps) rate:

$$\tilde{t} = \frac{i}{\text{tps}} \times \text{tps}_{\text{base}} + \delta \tag{3}$$

where i is the latent frame index, δ is a modality-specific offset (set to 15000 in practice), $\text{tps} = \text{fps}/\text{tcf}$ is tokens

per second (tps) at the modality's native rate, and $\text{tps}_{\text{base}} = 24/\text{tcf}$ is the reference rate, where tcf denotes the temporal compression factor from the video tokenizer. A 12 FPS video has frames spaced twice as far apart in position space as a 24 FPS video; a 48 FPS video has frames spaced half as far apart. The same formula aligns audio and action tokens temporally with the video stream. At inference, the FPS can be set explicitly to control the temporal dynamics of the generated output.

Modality margin. For larger dense variants of Cosmos3, we observed over-saturation and checkerboard artifacts in initial video frames. The cause: the last text token and the first vision token sit at adjacent temporal positions, giving them nearly identical temporal embeddings. We address this by inserting a fixed temporal gap k between the last text token and the first vision token, shifting all downstream vision, audio, and action positions uniformly. This creates a buffer in position space that gives the model a clearer text-to-vision transition signal without requiring architectural changes. In all our models, we use $k = 15000$.

2.3. Multi-modality

Cosmos3 processes four distinct modalities: text, video, action, and sound. This requires handling position embeddings properly to correctly align them. We do so by adapting FPS-modulated 3D RoPE for action and sound despite them being single dimensional token sequences.

2.3.1. Video

A world foundation model for physical AI must handle visual inputs across a wide range of spatial resolutions (256p to 720p), aspect ratios, and video durations — often within the same training batch. Processing raw pixels directly is computationally intractable at these scales; the transformer's attention cost grows quadratically with sequence length, making aggressive compression a prerequisite. At the same time, the tokenizer must preserve sufficient spatial and temporal fidelity for the model to reason about fine-grained visual dynamics. This calls for a tokenizer that compresses aggressively, operates causally so that encoding does not require access to future frames, and degrades gracefully across resolutions without requiring separate models or fixed output dimensions.

Tokenizer. Cosmos3 uses a causal video VAE its visual tokenizer to encode raw pixel inputs into a compact latent representation for the MoT transformer, and decodes generated latents back to pixel space. Images are handled uniformly as single-frame videos. Because the VAE applies fixed compression ratios rather than fixed output dimensions, the number of tokens scales naturally with input resolution and duration without any architectural changes. Temporal compression follows the causal formula $T_\ell = 1 + (T - 1)/4$, which applies uniformly to any duration and to single-frame images ($T_\ell = 1$). Multiple aspect ratios are supported out of the box, as the spatial grid dimensions adjust proportionally. During training, samples at different resolutions and durations are handled via sequence packing, filling each GPU's token budget with multiple variable-length sequences rather than padding to a fixed length.

Compression. The VAE applies a $16\times$ spatial compression and a $4\times$ temporal compression, yielding a 48-channel latent representation. Temporal compression is causal: given a video of T pixel frames, the number of latent frames is $T_\ell = 1 + (T - 1)/4$, where the first frame is encoded independently and subsequent frames are compressed in groups of four. On top of the VAE latents, the Cosmos3 MoT applies an additional 2×2 spatial patchification, bringing the total effective spatial compression to $32\times$ per dimension. Table 3 shows the resulting vision token counts at the three training resolutions.

Causal streaming encoding. The VAE encoder uses causal 3D convolutions throughout, padding only in the past-temporal direction. This enables chunk-by-chunk encoding: the encoder first primes on the initial frame to initialize its per-layer temporal caches, then processes the remaining frames in fixed-size chunks while carrying these caches forward. At no point does the encoder require access to future frames, and peak memory scales with chunk size rather than total video length. Decoding is similarly streaming, processing one latent frame at a time with cached state from prior frames.

Table 3: Vision token counts at standard training resolutions for 189 pixel frames ($T_\ell = 48$).

Resolution	Spatial grid (H×W)	Vision tokens
256p	8×14	5,376
480p	15×26	18,720
720p	22×40	42,240

Latent normalization and projection. The 48-channel latents are normalized per-channel using fixed mean and standard deviation statistics. These normalized latents are then patchified and projected into the Cosmos3 MoT's transformer's hidden dimension via a learned linear layer, and projected back out after generation via a symmetric decoder projection.

Generation modes. Given text tokens $\mathbf{T}$, clean vision tokens $\mathbf{V}$, and noisy vision tokens $\tilde{\mathbf{V}}$, where $\langle$EOS$\rangle$ marks the end of text and $\langle$BOV$\rangle$ marks the beginning of the generation split, Cosmos3 supports four video generation modes across varying resolutions, aspect ratios, durations, and frame rates:

- **Text-to-Image (T2I):** Image generation from a text prompt, treating the image as a single-frame video.

$$\mathbf{S}_{\text{T2I}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \tilde{\mathbf{V}}_0] \tag{4}$$

- **Text-to-Video (T2V):** Video generation of arbitrary duration and resolution from a text prompt.

$$\mathbf{S}_{\text{T2V}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \tilde{\mathbf{V}}_0, \ldots, \tilde{\mathbf{V}}_{T_v}] \tag{5}$$

- **Video-to-Video (I2V / V2V):** Video generation conditioned on a text prompt and either a single input image (I2V) or a full input video (V2V).

$$\mathbf{S}_{\text{I2V}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_0, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}] \tag{6}$$

$$\mathbf{S}_{\text{V2V}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_0, \ldots, \mathbf{V}_{T_c}, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}] \tag{7}$$

2.3.2. Text

[TODO: complete section.]

2.3.3. Sound

Cosmos3 processes audio using a dedicated audio VAE separate from the video tokenizer. Raw audio sampled at 48 kHz is encoded with a hop size of 1920 samples, yielding an effective latent rate of $\text{TPS}_{\text{audio}} = 48000/1920 \approx 25$ tokens per second. Because this rate differs significantly from the video latent rate of 6 TPS (24 FPS downsampled by factor 4), naive sequential indexing would produce a large and growing temporal gap between co-occurring video and audio tokens. FPS modulation (Eq. **??**) resolves this: at $t = 1.0$ s the video token at latent index 6 maps to effective position $6 \times (24/6) = 24.0$, while the audio token at latent index 25 maps to $25 \times (24/25) = 24$.

Token structure. Audio latents are reshaped to $(T, 1, 1)$, collapsing spatial dimensions so that the same 3D MRoPE layer used for video can be applied without modification. Audio tokens therefore carry temporal coordinates only ($h = w = 0$), consistent with the unified coordinate assignment described in Section 2.2.

Asymmetric embedding. Video and audio tokens share the same transformer backbone but must be distinguishable. Video tokens retain their original embedding (projection + timestep + positional encoding). Audio tokens receive an additional learnable Audio Modality Embedding vector summed into their embedding, providing a clear modality signal without modifying any pretrained video weights.

Token ordering and attention. Audio tokens are appended immediately after video tokens within the full-attention generation split, so the packed sequence takes the form `[Text (causal)] [Video latents + Audio latents (full attention)]`. Full bidirectional attention between video and audio tokens within this split enables cross-modal synchronization during joint generation.

Generation modes. Given text tokens $\mathbf{T}$, clean video tokens $\mathbf{V}$, noisy video tokens $\tilde{\mathbf{V}}$, clean audio tokens $\mathbf{U}$, and noisy audio tokens $\tilde{\mathbf{U}}$, where $\langle\text{EOS}\rangle$ marks the end of text and $\langle\text{BOV}\rangle$ marks the beginning of the generation split, the architecture supports five audio-specific modes by controlling which modality is noised (target) versus held clean (condition):

- **Text-to-Audio (T2A):** Text-to-audio generation.

$$\mathbf{S}_{\text{T2A}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \tilde{\mathbf{U}}_1, \ldots, \tilde{\mathbf{U}}_{T_a}] \tag{8}$$

- **Text-to-Audio-Video (T2AV):** Joint text-to-audio-and-video generation.

$$\mathbf{S}_{\text{T2AV}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}, \tilde{\mathbf{U}}_1, \ldots, \tilde{\mathbf{U}}_{T_a}] \tag{9}$$

- **Video-to-Audio-Video (TV2A, Foley):** Audio generation conditioned on a clean video and text prompt.

$$\mathbf{S}_{\text{TV2A}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_1, \ldots, \mathbf{V}_{T_v}, \tilde{\mathbf{U}}_1, \ldots, \tilde{\mathbf{U}}_{T_a}] \tag{10}$$

- **Text-Audio-to-Video (TA2V):** Video generation conditioned on a clean audio track and text prompt.

$$\mathbf{S}_{\text{TA2V}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}, \mathbf{U}_1, \ldots, \mathbf{U}_{T_a}] \tag{11}$$

- **Text-to-Video (T2V):** Standard text-to-video with audio disabled, retaining full backward compatibility.

$$\mathbf{S}_{\text{T2V}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}] \tag{12}$$

2.3.4. Action

Action tokens are a first-class modality in Cosmos3, enabling the model to function as a world simulator that can reason about and generate physical interactions across diverse embodiments. Cosmos3 targets four domains: camera motion, robotic manipulation, human motion (whole body and hand), and autonomous vehicles. Because these embodiments have fundamentally different action spaces — joint angles, end-effector poses, waypoints, and steering commands, among others — a unified architecture must handle heterogeneous control signals while sharing a common transformer backbone.

Token structure. Action tokens are continuous vectors representing embodiment-specific control signals at each timestep. They carry temporal coordinates only ($h = w = 0$), consistent with the unified coordinate assignment described in Section 2.2. Temporal alignment with the video stream is achieved via FPS modulation (Eq. **??**). As a concrete example, at $f = 16$ FPS with a video temporal compression factor of 4, vision tokens have TPS $= 4$ while action tokens have TPS $= 16$, producing the following scaled temporal positions:

Modality	TPS ρ	Token Indices	Scaled Times
Vision	4	$[0, 1]$	$[0, 6]$
Action	16	$[0, 1, 2, 3]$	$[1.5, 3.0, 4.5, 6.0]$

Domain-aware projection. Different embodiments operate in incompatible action spaces, making a single shared projection head insufficient. Cosmos3 uses a domain-aware linear layer that maintains separate weight matrices per embodiment domain while sharing all other transformer parameters. For an input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ and

domain identifier $k \in \{1, \ldots, K\}$, the projection is:

$$f_k(\mathbf{x}) = \mathbf{W}_k \mathbf{x} + \mathbf{b}_k \tag{13}$$

where $\mathbf{W}_k \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and $\mathbf{b}_k \in \mathbb{R}^{d_{\text{out}}}$ are domain-specific weights and biases.

Generation modes. Given text tokens $\mathbf{T}$, clean vision tokens $\mathbf{V}$, noisy vision tokens $\tilde{\mathbf{V}}$, clean action tokens $\mathbf{A}$, and noisy action tokens $\tilde{\mathbf{A}}$, Cosmos3 supports four action-conditioned generation modes, sampled equally during training:

- **Video Model:** Predicts future video frames from an initial conditioning frame. No action tokens are present:
$$\mathbf{S}_{\text{video}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_0, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}] \tag{14}$$
- **Forward Dynamics:** Predicts future video frames conditioned on both an initial frame and a sequence of actions:
$$\mathbf{S}_{\text{forward}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_0, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}, \mathbf{A}_1, \ldots, \mathbf{A}_{T_a}] \tag{15}$$
- **Inverse Dynamics:** Predicts the action sequence that produced an observed video trajectory:
$$\mathbf{S}_{\text{inverse}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_0, \mathbf{V}_1, \ldots, \mathbf{V}_{T_v}, \tilde{\mathbf{A}}_1, \ldots, \tilde{\mathbf{A}}_{T_a}] \tag{16}$$
- **Policy Mode:** Jointly predicts future video frames and actions, enabling end-to-end policy learning:
$$\mathbf{S}_{\text{policy}} = [\mathbf{T}, \langle\text{EOS}\rangle, \langle\text{BOV}\rangle, \mathbf{V}_0, \tilde{\mathbf{V}}_1, \ldots, \tilde{\mathbf{V}}_{T_v}, \tilde{\mathbf{A}}_1, \ldots, \tilde{\mathbf{A}}_{T_a}] \tag{17}$$

2.3.5. Multimodal Sequence Packing

Cosmos3 packs all modalities — text, video, action, and sound — into a single flat token sequence per training sample. This sequence is split into two attention regions: a causal split for text tokens (which attend only to preceding text, preserving the autoregressive behavior of the language backbone) and a full split for all generation tokens (which attend bidirectionally to one another and to the entire text context). Special delimiter tokens mark the boundary between the two regions. Whichever modalities are present in a given sample are concatenated into the full split; absent modalities are simply omitted, so the same packing machinery handles text-to-video, image-to-video, robotics, and audio generation. Because training batches mix samples of varying resolution, aspect ratio, and duration, multiple variable-length samples are packed into a single GPU sequence up to a fixed token budget, and the runtime separates causal and full tokens into contiguous buffers for efficient attention. A per-sample sequence plan controls which frames within each modality are clean conditioning inputs versus noised generation targets, unifying all task modes under one sequence packing infrastructure.

3. Data

3.1. Pre-training Data Curation

3.2. Mid-training Data Curation

3.3. Post-training Data Curation

4. Training

4.1. Training Recipe

4.1.1. Pre-training

Pre-training teaches the model to jointly generate images and videos across a wide range of resolutions and generation modalities. The recipe is designed around three pillars: multi-resolution training, unified generation modes (T2I, T2V, and I2V), and dynamic FPS conditioning.

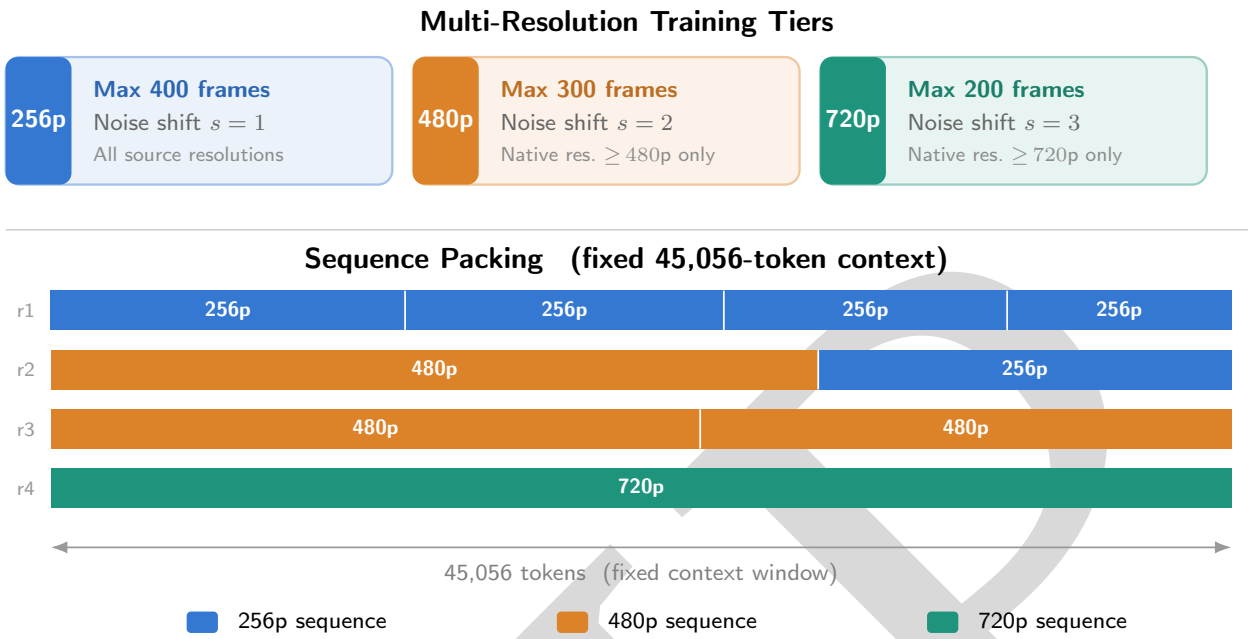

Figure 4: **Multi-resolution training and sequence packing.** *Left:* The three resolution tiers (256p, 480p, 720p) differ in their maximum frame budget, eligible source material, and rectified-flow noise-shift value. *Right:* Variable-length sequences from different tiers are packed together to fill a fixed 45,056-token context window, maximising GPU utilisation without padding.

Multi-resolution Training. Rather than committing to a single output resolution, we train simultaneously across three resolution tiers—256p, 480p, and 720p—allowing the model to develop resolution-agnostic representations while still being exposed to high-fidelity content. The training data is partitioned accordingly: the 256p stream draws from the full dataset (all native resolutions are eligible), the 480p stream is restricted to source material with native resolution at or above 480p, and the 720p stream uses only content at or above 720p, preserving sharpness and fine detail at the highest tier. Each resolution tier imposes a different maximum frame budget to keep GPU memory usage roughly constant across resolutions: up to 400 frames at 256p, 300 frames at 480p, and 200 frames at 720p. Batch composition across the three tiers follows a 1:1:3 ratio (image, video-256p, video-480&720p), biasing the model toward higher-resolution video while retaining sufficient low-resolution signal for stable early convergence.

To prevent gratuitous recompilation overhead while supporting variable sequence lengths, we use token packing with a fixed budget of 45,056 tokens per sequence. Shorter sequences at lower resolutions are packed together to fill each batch, maximizing GPU utilization without padding. Following Waver **?**, we use logitnormal noise distribution for image batches and mode sampling for video batches. We found that the use of mode sampling results in better generation quality. We also use resolution-adaptive shift values: $s = 1$ at 256p, $s = 2$ at 480p, and $s = 3$ at 720p.

Training Modes. A single pretraining run jointly covers three generation tasks, distinguished only by the number of conditioned visual frames prepended to each video sample.

Text-to-Image (T2I). All image samples are trained with zero conditioned frames: the model must hallucinate the entire visual content from text alone. Images are drawn at all three resolution tiers with a mixture of long descriptive captions (70%), medium-length descriptive captions (20%), and short captions (10%), broadening the range of language grounding.

Text-to-Video (T2V). For video samples, the majority (70%) are also assigned zero conditioned frames, making the model learn to generate a full video clip from a text prompt. As with images, captions are sampled from

the same three-level distribution. The model additionally receives duration, FPS, and timestamp metadata appended to the text prompt, training it to generate videos of specified length and temporal density.

Image-to-Video (I2V). The remaining video samples are split between single-frame conditioning (20%) and two-frame conditioning (10%). In the single-frame case ($k = 1$) the first latent frame is held clean and only the subsequent frames are noised, training the model to animate a given still image. In the two-frame case ($k = 2$) the first two latent frames are conditioned, which encourages the model to respect an initial motion direction and is useful for interpolation and continuation tasks. This mixed conditioning strategy is implemented via a `SequencePlanAugmentor` that samples k at training time from the distribution $\{0{:}0.7,\ 1{:}0.2,\ 2{:}0.1\}$, requiring no architecture changes—clean conditioned frames simply occupy a distinct segment of the packed token sequence with their noise level fixed at zero.

FPS Modulation. All video streams are loaded with dynamic FPS sampling (`use_dynamic_fps=True`). The physical temporal spacing between tokens therefore varies across samples: a clip sampled at 30 FPS packs frames more densely in real time than the same number of tokens sampled at 16 FPS. The 3D MRoPE position encodings reflect this: temporal coordinates are assigned in proportion to real-world time rather than token index (see Sec. 2), with a base rate of 24 FPS. Duration and FPS information are also appended to the text prompt, so the model can be conditioned on specific temporal characteristics at inference time.

Optimization. Only the generation-specific parameters are updated during pretraining: the MoE generation experts (`moe_gen`), the noise-level time embedder, and the VAE↔LLM projection layers (`vae2llm`, `llm2vae`). The VLM backbone weights remain frozen, preserving the language and visual understanding capabilities inherited from Qwen3-VL **?**. We use FusedAdamW with learning rate 10^{-4}, $(\beta_1, \beta_2) = (0.9, 0.99)$, weight decay 0.05, and gradient clipping at norm 1.0. The learning rate follows a cosine schedule decaying from a peak of $0.95\times$ to a floor of $0.30\times$ over 100k iterations, without a warm-up phase (the model is initialized from a prior checkpoint). Classifier-free guidance training uses a text-dropout rate of 10% across all modalities.

Tokens Seen: We train 8B model for a total of 24.3T tokens on 1024 GB200 GPUs.

4.1.2. Mid-Training

Mid-training bridges the gap between broad pretraining and downstream deployment by simultaneously expanding the model's modality coverage and sharpening its performance in domain-critical scenarios. The stage is characterized by two complementary objectives: *multi-modal integration*, which adds audio, action, and interleaved generation to the existing image/video capabilities; and *domain specialization*, which introduces curated datasets for robotics, autonomous vehicles (AV), and synthetic data generation (SDG).

Staged Modality Introduction. Rather than adding all new modalities at once, we adopt a sequential four-stage curriculum to avoid distributional shock to the pretrained model. Each stage activates one additional modality while retaining all prior ones: Stage 1 refines image and video quality with new high-quality data; Stage 2 adds audio with text-to-video-and-sound generation; Stage 3 introduces action conditioning (action-to-video, video-to-action, and text-to-video-with-action modes); and Stage 4 adds interleaved generation (image editing and image transfer). The modality mixing ratios are adjusted at each stage to accommodate the new data streams while preserving visual generation quality (see **??**).

Data. For image data, the mid-training mixture combines the pretraining dataset with $\sim$1M high-quality real images, 3M high-quality synthetic images, and text-rendering data. For video, the pretraining stream is augmented with $\sim$2.8M high-quality real videos (emphasizing AV, human, and robotics content), $\sim$4.2M capability-focused clips (faces, hand gestures, sports, and human–object interaction), and $\sim$900K synthetic data generation (SDG) videos from DriveSim and Isaac Sim. Audio, action, and interleaved modalities draw from filtered Cosmos pretraining audio, robotics/AV/camera-pose datasets, and Cosmos-Transfer/ChronoEdit sequences, respectively.

Optimization. The mid-training optimization hyperparameters mirror pretraining: FusedAdamW with learning rate 10^{-4}, weight decay 0.05, gradient clipping at norm 1.0, and loss scale 10. The LambdaLinear learning rate schedule is retained, and the context window remains fixed at 45,056 tokens with generation resolutions spanning 256p–720p.

4.2. Training Optimizations

Training Cosmos3 at scale requires addressing memory, throughput, and hardware utilization challenges across both dense and MoE model variants. We describe the key system-level optimizations that collectively enable efficient large-scale training.

Hybrid Sharding Data Parallel (HSDP). We adopt Hybrid Sharding Data Parallel Hao et al. (2024), a PyTorch FSDP-based strategy that balances the memory efficiency of Fully Sharded Data Parallel (FSDP) with the communication efficiency of Distributed Data Parallel (DDP). Tuning `data_parallel_shard_degree` and `data_parallel_replicate_degree` for each model size enables efficient utilization of our GPU topology.

Torch Compile. We apply `torch.compile` with `fullgraph=True` and `dynamic=True` across the training graph. The `fullgraph` mode eliminates CPU overheads and enables operator fusion, while `dynamic=True` handles the variable sequence lengths arising from mixed-modality batches — where, for example, video generator pathway tokens are substantially longer than those of image batches across iterations. Torch compile improves training throughput by **20–41%** across Cosmos3 model sizes.

MoT Attention. Cosmos3's Mixture-of-Transformers architecture requires causal attention for the reasoner pathway (attending only to reasoner tokens) and full bidirectional attention for the generator pathway (attending to both reasoner and generator tokens). Handling these dual-mask requirements efficiently with standard attention implementations leads to non-optimal hardware utilization. We co-designed and implemented a custom two-way flat attention mechanism that concatenates tokens from both pathways in an interleaved manner and dispatches a separate SDPA module per pathway, implemented with a Cutlass backend. This attention kernel handles both causal and bidirectional mask requirements with variable-length inputs, improving training throughput by **22%**.

MoE Kernel Optimizations. Cosmos3 supports both dense (Cosmos3-Nano, Cosmos3-Super) and Mixture-of-Experts (Cosmos3-Ultra) configurations. For MoE training, we implemented Grouped GEMM kernels to accelerate expert computation, reduced the overhead of token-to-expert and expert-to-token permutation operations, fused score weighting into the SwiGLU activation to eliminate its independent overhead, and tuned a load-balancing auxiliary loss to maintain balanced expert utilization. Expert parallelism is implemented alongside HSDP; while HSDP remains marginally faster for large sequence lengths due to all-to-all overhead and load imbalance, the combined set of MoE optimizations improves training throughput by $>$**10**$\times$ relative to the unoptimized baseline without impacting model quality.

Context Parallelism. To scale training to sequences exceeding 100K tokens, we employ a Ulysses-style context parallelism (CP) scheme Jacobs et al. (2023). Leveraging our packed sequence format, data is partitioned across CP ranks while retaining the global metadata required for efficient post-all-to-all attention mask generation. This pipeline is also extended to inference to handle long-context video batches.

MXFP8 Training. To leverage the native quantization capabilities of NVIDIA Blackwell GPUs, all linear layers in both the reasoner and generator pathways are trained in MXFP8 precision. Interface modules — LLM2VAE, the time embedder, and VAE2LLM — retain full BF16 precision to preserve numerical stability at modality boundaries.

Resolution-Synchronized Data Processing. Mixed-resolution video training introduces severe load imbalances across FSDP ranks due to token-count disparities between resolution buckets. We address this with a globally-seeded joint dataloader that groups samples into resolution-specific streams, ensuring all ranks implicitly

Table 4: Training throughput for Cosmos3 dense models. TFLOPS and MFU are reported per GPU; token throughput is decomposed into image and video token streams.

Model	Iter (s)	TFLOPS	MFU	Iter/hr	Img Tok/hr/GPU (M)	Vid Tok/hr/GPU (M)
Cosmos3-Nano	7.1	520	0.23	507	4.56	16.23
Cosmos3-Super	19.5	673	0.30	185	1.66	5.91

process the same resolution bucket per iteration. Combined with CUDA graph capture, this eliminates per-step recompilation overhead and enables efficient large-scale pretraining.

Asynchronous Checkpointing. Checkpoint saves are fully overlapped with training using a Gloo process group to avoid interference with the NCCL training communicator, inspired by Torchtitan's reference implementation Liang et al. (2024). Save plans are computed once on the first checkpoint and reused thereafter, eliminating per-checkpoint metadata communication across ranks. Since checkpoint intervals are approximately one hour and object store write times are highly variable (ranging from minutes to 30 minutes for large models), asynchronous checkpointing eliminates this overhead almost entirely at the cost of a small increase in host memory.

Throughput Summary. Table 4 reports steady-state training statistics for the Cosmos3 dense model configurations.

5. Inference

5.1. Inference Ablations and Settings

Recommended Default Generation Settings.

Duration-FPS Template.

Resolution Template.

Negative Prompt.

5.2. Inference Optimizations

6. Results

6.1. Image and Video Generation Benchmarks

We evaluate Cosmos3 across two complementary automated benchmarks and a human evaluation protocol, jointly measuring perceptual video quality, physical plausibility, and embodied task correctness.

6.1.1. Automated Evaluation

PAIBench-G. PAIBench-G Zhou et al. (2025) is the video generation track of the Physical AI Benchmark, designed to measure whether video generation models can produce outputs that are both visually compelling and physically plausible in real-world Physical AI scenarios. It comprises 1,044 image–text prompt pairs drawn from six Physical AI domains: Human (299 videos), Autonomous Vehicle (239), Common Sense (174), Robotics (107), Physics (107), and Industry (107). The overall score is an equal-weight combination of a **Quality Score** and a **Domain Score**:

$$\text{Overall} = 0.5 \times \text{Quality Score} + 0.5 \times \text{Domain Score}. \tag{18}$$

The Quality Score follows the VBench/VBench++ evaluation protocol Huang et al. (2024a); **?**. Six metrics apply to all generations: Subject Consistency (SC, DINO-based cross-frame similarity), Background Consistency (BC, CLIP-based cross-frame similarity), Motion Smoothness (MS, frame-interpolation MAE), Aesthetic Quality (AQ, LAION predictor), Imaging Quality (IQ, MUSIQ perceptual predictor), and Overall Consistency (OC,

Table 5: PAIBench-G Image-to-Video results (Qwen2.5-VL-72B judge). Overall $= 0.5\times$Quality $+0.5\times$Domain. Ground-truth videos provide a reference ceiling.

Model	Overall	Domain	Quality
Ground Truth	**82.6**	87.1	78.0
Cosmos3 32B (Ours)	82.5	87.3	77.8
Cosmos3 8B (Ours)	78.6	80.8	76.5
Veo-3.1	**82.6**	**87.6**	77.6
HunyuanVideo-1.5	81.7	85.9	77.6
Wan2.2-A14B	81.3	85.3	77.3
Cosmos-Predict2.5-2B	81.2	84.6	77.9
Cosmos-Predict2.5-14B	81.1	84.0	**78.1**
Wan2.2-5B	81.0	84.6	77.4
Wan2.1-I2V-14B	81.0	84.6	77.3

Table 6: PAIBench-G Text-to-Video results (Qwen2.5-VL-72B judge). Overall $= 0.5\times$Quality $+0.5\times$Domain.

Model	Overall	Domain	Quality
Cosmos3 32B (Ours)	**79.2**	**85.4**	**72.9**
Cosmos3 8B (Ours)	78.6	84.6	72.7
Veo-3.1	79.1	85.2	72.9
Wan2.2-A14B	78.0	83.2	72.8
Seedance-1.5-Pro	76.9	82.1	71.6
HunyuanVideo-1.5	76.5	80.9	72.0
Cosmos-Predict2.5-2B	76.5	79.9	73.2
Cosmos-Predict2.5-14B	76.4	79.5	73.2
Wan2.1-14B	76.4	80.1	72.7
Wan2.2-5B	76.3	79.6	73.0

ViCLIP video–text alignment)—the only metric that incorporates the prompt text. Two additional metrics specifically evaluate image-to-video conditioning: I2V Subject (IS, DINO similarity between the conditioning frame and generated frames) and I2V Background (IB, DreamSim similarity between the conditioning frame and generated frames). The Domain Score quantifies physical and semantic accuracy through an MLLM-as-Judge framework: domain-specific YES/NO verification questions are evaluated per video at 2 frames per second, and accuracy is averaged across domains weighted by domain video count. The benchmark achieves a Pearson correlation of $r = 0.918$ with human ELO rankings Zhou et al. (2025), validating its alignment with human judgment.

Tables 6 and 5 report text-to-video and image-to-video results on PAIBench-G evaluated with Qwen2.5-VL-72B-Instruct as the VLM judge.

RBench. RBench Deng et al. (2026) evaluates video generation in embodied task scenarios, emphasizing task correctness and physical plausibility in robot–object interactions over purely perceptual realism. The benchmark comprises 650 image–text evaluation cases from two complementary splits: 250 task-oriented pairs spanning five categories (Common Manipulation, Long-horizon Planning, Multi-entity Collaboration, Spatial Relationship, and Visual Reasoning; 50 samples each), and 400 embodiment-specific pairs covering four robot morphologies (Dual-arm, Humanoid, Single-arm, and Quadruped; 100 samples each). Prompts are sourced from RoVid-X Deng et al. (2026), a curated corpus of 4M robotics clips spanning over 1,300 skills at 720p/30fps.

Table 7: RBench image-to-video results. Score is the mean of TC and VQ across all 650 evaluation cases.

Model	Type	Score
Cosmos3 32B (Ours)	Open-source	55.2%
Cosmos3 8B (Ours)	Open-source	–
Wan 2.6	Commercial	**60.7**%
Seedance 1.5 Pro	Commercial	58.4%
Wan 2.5	Commercial	57.0%
Hailuo v2	Commercial	56.5%
Veo 3	Commercial	56.3%
Wan2.2-A14B	Open-source	50.7%
Cosmos-Predict2.5	Robotics-specific	46.4%
HunyuanVideo 1.5	Open-source	46.0%

Each generated video is scored on Task Completion (TC) and Visual Quality (VQ):

$$\text{TC} = \frac{\text{PSS} + \text{TAC}}{2}, \qquad \text{VQ} = \max\Big(0,\ 0.8 \cdot \text{RSS} + 0.2 \cdot \text{MS} - \text{Pen}_{\text{MA}} - \text{Pen}_{\text{RSS}}\Big). \tag{19}$$

TC averages Physical-Semantic Plausibility (PSS) and Task-Adherence Consistency (TAC). PSS uses MLLM-as-Judge evaluation to detect violations of physical and commonsense constraints—including object interpenetration, floating or unsupported objects, incorrect grasping or contact, and object duplication or disappearance. TAC measures instruction adherence via task responsiveness and key action completeness, computed as a normalized fraction of completed sub-steps. VQ is a penalized combination of Robot–Subject Stability (RSS, contrastive MLLM checks for consistent robot morphology and object identity across frames) and Motion Smoothness (MS, temporal anomaly detection). The penalty terms Pen_{MA} and Pen_{RSS} deduct points for overly static outputs and for catastrophic failures in robot structure or object identity, respectively. The final model score is the mean of TC and VQ across all 650 cases; RBench achieves a Spearman correlation of $\rho = 0.96$ with human judgments across 25 evaluated models Deng et al. (2026).

Table 7 summarizes results. The leading general-purpose model, Wan 2.6, achieves 60.7%, while all models fall substantially short of human-level performance, illustrating the difficulty of embodied world simulation.

6.1.2. Human Evaluation

Automated metrics offer scalability but cannot fully capture subtle aspects of prompt adherence, physical plausibility, and subjective visual quality that are critical for Physical AI applications. We therefore complement automated benchmarking with Cosmos HUE (Human Understanding Evaluation), an internal human scoring protocol grounded in the same PAI-Bench-G prompt set.

Cosmos HUE. Each evaluation batch covers 100 prompts, with five random-seed generations produced per model per prompt. Prompts are sampled from the PAI-Bench-G video set and annotated with human questions derived from the corresponding ground-truth videos. The domain distribution mirrors PAIBench-G: Human (29%), Robotics (17%), Common Sense (14%), Autonomous Vehicle (11%), Industry (10%), Physics (10%), and Miscellaneous (9%). Annotators score each generation on the same two axes as PAIBench-G—visual quality and domain correctness—enabling direct cross-evaluation comparison and exposing systematic gaps between automated metrics and human perception.

6.1.3. Core Results

Scaling Gains.

Table 8: Cosmos HUE human evaluation results.

Model	T2V Score	I2V Score
Ground Truth	**0.917**	–
Cosmos3 32B (Ours)	**–**	**–**
Cosmos3 8B (Ours)	**–**	**–**
Veo-3	0.880	–
Seedance-1.5-Pro	0.850	**0.826**
Wan2.2-A14B	0.841	0.803
HunyuanVideo-1.5	0.789	0.803
Wan2.1-14B	0.794	0.689
Cosmos-Predict2.5-14B	0.768	0.718
Cosmos-Predict2.5-2B	0.738	0.728
Wan2.2-TI2V-5B	0.742	0.660

6.1.4. Observations

6.2. Audio Generation Benchmarks

Generating video and audio concurrently introduces the complex challenge of cross-modal alignment. We evaluate Text-to-Audio-Visual (T2AV) model quality across four interdependent dimensions: visual quality and temporal consistency, acoustic fidelity, audio-visual alignment and spatial coherence, and joint prompt following. We define a dedicated benchmark suite, **Cosmos-SoundBench**, and combine objective statistical metrics with Multimodal Large Language Model (MLLM) judges and specialized audio-visual interaction models.

6.2.1. Cosmos-SoundBench

Cosmos-SoundBench-v1 comprises 200 evaluation prompts drawn from two complementary sources: 140 non-speech prompts from FoleyBench **?** covering ambient sounds, impacts, and environmental effects, and 60 speech prompts from VABench **?** targeting talking-head and dialogue scenarios. A more comprehensive v2 dataset of 150 video–audio–caption tuples with richer diversity metadata (gender, demographics, expressivity, multilingual coverage) is under development.

6.2.2. Automated Evaluation

Joint Prompt Score (JPS). Because a T2AV prompt dictates both visual action and acoustic environment, we employ Gemini-3-Flash **?** as an MLLM judge capable of parsing both video frames and audio tracks simultaneously. The judge decomposes each prompt into individual visual and audio elements, evaluates each for unambiguous presence using strict boolean checks, and produces six scores on a $[1, 10]$ scale: Video Instruction Following (Video-IF), Audio Instruction Following (Audio-IF), joint Audio-Visual Instruction Following (AV-IF), Audio-Visual Consistency (AV-Con), Visual Quality, and Audio Quality. The Joint Prompt Score is the average of these six sub-scores:

$$\text{JPS} = \frac{1}{6}\big(\text{Video-IF} + \text{Audio-IF} + \text{AV-IF} + \text{AV-Con} + \text{Visual} + \text{Audio}\big). \tag{20}$$

Table 9 reports JPS results on Cosmos-SoundBench-v1. Cosmos sound midtrain improves over the pretrained checkpoint across all sub-scores, narrowing the gap to commercial models.

Acoustic Quality (AQ). We evaluate the generated audio track in isolation using AGAV-Rater **?**, a VideoLLaMA2-based regression model trained on human Mean Opinion Scores for AI-generated audio-visual content. AGAV-Rater produces three continuous scores: Audio Quality (AQ), measuring perceived fidelity and absence of artifacts; Audio-Visual Consistency (AVC), measuring semantic coherence between audio and visuals; and Overall Audio-Visual Quality (OAVQ), a joint quality rating.

Table 9: Joint Prompt Score (JPS) results on Cosmos-SoundBench-v1. All sub-scores are on a $[1, 10]$ scale; higher is better.

Model	Video-IF	Audio-IF	AV-IF	AV-Con	Visual	Audio	JPS
Sora 2.0	9.75	9.82	9.74	9.02	7.92	8.59	**9.14**
Veo 3.1	9.74	9.57	9.66	8.58	6.96	8.30	8.80
Veo 3.0	9.72	9.48	9.63	8.63	7.48	8.34	8.88
LTX-2	8.67	8.64	8.63	8.18	7.23	7.60	8.16
Cosmos sound midtrain	8.64	8.10	8.25	8.02	6.70	7.81	7.92
Cosmos sound pretrain	8.73	6.06	7.34	5.77	6.40	5.73	6.67

Table 10: AGAV-Rater scores on Cosmos-SoundBench-v1. Higher is better.

Model	Audio Quality	AV Consistency	Overall Quality
Veo 3.1	**26.16**	**33.81**	**31.65**
Veo 3.0	25.68	32.33	28.34
Sora 2.0	15.29	23.43	18.15
LTX-2	−7.94	−1.55	−3.28
Cosmos sound midtrain	−3.64	8.59	4.87
Cosmos sound pretrain	−13.04	−21.85	−14.75

Table 10 reports AGAV-Rater scores. Cosmos sound midtrain shows substantial improvement over the pretrained model on audio-visual consistency, indicating that mid-training specifically enhances cross-modal alignment.

Lip Sync (SyncNet). For speech prompts involving talking heads, we measure lip-speech synchronization using SyncNet **?**, which encodes lip crops and aligned MFCC windows into a shared embedding space and searches over a ± 15 frame window for the best-aligned offset. The key metric is **Lip Sync Confidence**: the sharpness of the synchronization peak across offsets, where higher values indicate clearer correlation between lip movements and speech. A confidence score below 3 is considered a failure.

Table 11 reports lip sync results on the 60 speech prompts. Commercial models achieve substantially higher face detection rates and sync confidence, highlighting lip synchronization as a key area for improvement in Cosmos sound models.

6.2.3. Overall T2AV Score

To produce a single leaderboard metric that balances prompt adherence, raw generation quality, and cross-modal alignment, we define:

$$\text{Overall T2AV Score} = 0.35 \cdot \text{JPS} + 0.25 \cdot \text{VQ} + 0.20 \cdot \text{AQ} + 0.20 \cdot \text{AVS}, \tag{21}$$

where VQ is the visual quality score (shared with the video evaluation pipeline), AQ is the AGAV-Rater audio quality, and AVS is a normalized audio-visual synchronization scalar inversely proportional to the AV temporal offset.

6.2.4. Core Results

Scaling Gains.

6.2.5. Observations

6.3. Action Generation Benchmarks

We evaluate the action capabilities acquired during Cosmos3 video-action mid-training across three complementary tasks: forward dynamics (predicting future video from an image and action sequence), inverse dynamics

Table 11: Lip sync evaluation on Cosmos-SoundBench-v1 speech prompts (60 total). Face Detection Rate indicates the fraction of prompts where a face was successfully detected and tracked.

Model	Face Detection Rate	Lip Sync Confidence
Veo 3.1	100% (60/60)	**5.20**
Veo 3.0	100% (60/60)	5.04
Sora 2.0	92% (55/60)	3.93
LTX-2	67% (40/60)	3.30
Cosmos sound midtrain	73% (44/60)	0.39
Cosmos sound pretrain	72% (43/60)	0.57

Table 12: Forward dynamics evaluation ($I + A \rightarrow V$). Reconstruction PSNR (dB) across datasets; higher is better. Navigation uses unseen data and unseen environments.

Model	DROID	AgiBotGEAR	UMI	Bridge	Navigation	Egocentric	AV
Cosmos3 (Ours)	–	–	–	–	–	–	–

(inferring actions from observed video), and policy (jointly generating video and actions from an image). The goal is to assess whether the mid-trained base model can be efficiently adapted—with a short post-training of approximately 4,000 steps at batch size 1,024 ($\sim$4M effective samples)—into a state-of-the-art specialized model. During post-training, the model is specialized to operate in a single mode, moving away from the mixed-mode training used in mid-training.

6.3.1. Forward Dynamics ($I + A \rightarrow V$)

We evaluate the model's ability to predict future video frames given an initial image and an action sequence. Three metrics capture complementary aspects of generation quality: **Reconstruction PSNR**, the standard peak signal-to-noise ratio between predicted and ground-truth video frames; **Action-Following (Dynamic PSNR)**, which computes PSNR only on pixel regions containing robot movement to prevent static background regions from dominating the score; and **Physics-Following PSNR**, which evaluates physical plausibility of object interactions after gripper contact (reported only on DROID).

Table 12 reports forward dynamics results across seven evaluation datasets spanning robotics manipulation (DROID, AgiBotGEAR, Bridge), camera-pose-conditioned generation (UMI, Navigation), egocentric hand tracking, and autonomous vehicle route prediction. These datasets collectively cover both in-domain and new-domain control modalities, with evaluation samples ranging from 100 (Navigation) to 6,243 (DROID).

6.3.2. Inverse Dynamics ($V \rightarrow A$)

We evaluate the model's ability to infer actions from observed video. Evaluation is conducted on non-overlapping chunks of 16 frames (e.g., frames 0–15, 16–31). The primary metric for robot action domains is **MSE** between predicted and ground-truth action vectors. For camera-pose and autonomous vehicle domains, we additionally report three pose-specific metrics: **Absolute Translation Error (ATE)**, the L2 distance between inferred and ground-truth camera translations; **Relative Translation Error (RTE)**, the L2 distance between inferred and ground-truth relative translations in the local frame; and **Relative Rotation Error (RRE)**, the angular distance between inferred and ground-truth relative rotation matrices, computed as $\text{RRE}_i = \arccos\big((\text{tr}(\mathbf{R}_{\text{error}}) - 1)/2\big)$. Predicted and ground-truth camera pose sequences are normalized to their first frame; when scales differ, Umeyama alignment is applied.

Table 13 reports inverse dynamics results across four evaluation datasets.

Table 13: Inverse dynamics evaluation ($V \rightarrow A$). Action MSE (↓) for robot action domains; ATE/RTE/RRE (↓) additionally reported for camera-pose domains.

Model	AgiBotGEAR	UMI	Navigation	Hand Pose
Cosmos3 (Ours)	–	–	–	–

Table 14: Policy evaluation ($I \rightarrow V + A$). Success Rate (%) reported for simulation benchmarks; Min L1 Error (↓) for AV open-loop evaluation. Prior methods are compared by effective post-training sample size.

Model	Eff. Samples	LIBERO	LIBERO-Gen	SimplerEnv	AV
Cosmos3 (Ours)	∼4M	–	–	–	–
X-VLA **?**	7.6M	–	–	–	–
GR00T **?**	2M–61M	–	–	–	–

6.3.3. Policy ($I \rightarrow V + A$)

For the policy task, the model jointly generates video and actions from an initial image, and we evaluate both task efficacy and video-action consistency. The primary metrics are: **Min L1 Error**, the minimum L1 action and video error across multiple generation seeds; **Success Rate**, the fraction of episodes in which the generated action sequence completes the specified task (evaluated via simulation rollout in SimplerEnv **?** and LIBERO **?**); and **Consistency (Action MSE)**, measuring agreement between predicted actions and actions inferred by an inverse dynamics model given the predicted video, serving as an internal coherence check.

We evaluate on four benchmarks: LIBERO (10.9 training hours, 10 episodes per task) is used primarily for debugging as the benchmark has been largely saturated; LIBERO generalization tests instruction-level generalization with new instructions on the same environment; SimplerEnv-Bridge (105 training hours, 24 episodes per task) trains on real images and evaluates in synthetic environments, providing a pseudo-in-domain transfer test; and AV (2,495 training hours, 1,000 evaluation episodes) is evaluated in an open-loop setting.

Table 14 reports policy evaluation results.

6.3.4. Core Results

Scaling Gains.

6.3.5. Observations

7. Applications

8. Related Work

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. 7

[2] Yufan Deng, Zilin Pan, Hongyu Zhang, Xiaojie Li, Ruoqing Hu, Yufei Ding, Yiming Zou, Yan Zeng, and Daquan Zhou. Rethinking video generation model for the embodied world. *arXiv preprint arXiv:2601.15282*, 2026. 17, 18

[3] Yanqi Hao, Zhiquan Lai, Wei Wang, Shengwei Li, Weijie Liu, Keshi Ge, and Dongsheng Li. Hsdp: Accelerating large-scale model training via efficient sharded data parallelism. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, pages 116–125. IEEE, 2024. 15

[4] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. Vbench: Comprehensive benchmark suite for video generative models. In *CVPR*, 2024a. 16

[5] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023. 15

[6] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, et al. Torchtitan: One-stop pytorch native solution for production ready llm pre-training. *arXiv preprint arXiv:2410.06511*, 2024. 16

[7] Fengzhe Zhou, Jiannan Huang, Jialuo Li, Deva Ramanan, and Humphrey Shi. PAI-Bench: A comprehensive benchmark for physical AI. *arXiv preprint arXiv:2512.01989*, 2025. 16, 17